

文件查找、文本筛选与批量处理

从目录树对象到安全 `argv`：把文件集合、文本记录、字段和批量副作用放进同一条证据链。

对象模型操作语义验证诊断经典任务

大号阅读版

本章阅读导航

先抓住一条主线：先从目录树中选择正确的文件集合，再把文件内容视为记录与字段，最后把每个路径安全地交给批量命令，并用独立证据证明没有漏改和误伤。

专题地图

- 知识专题 从目录树对象到记录流
- 知识专题 `find` 的搜索空间与可见性
- 操作专题 把任务要求翻译成 `find` 谓词
- 操作专题 表达式、逻辑与动作
- 知识专题 `locate` 与索引时间边界
- 知识专题 正则与 `grep` 记录选择
- 操作专题 字段、排序、去重与计数
- 操作专题 `sed`、`awk` 与结构化转换
- 操作专题 `NUL`、`xargs` 与批量 `argv`
- 诊断专题 空集合、过宽集合与部分失败

阅读时持续回答

1. 当前选择的是文件对象、路径名称，还是文件内容中的记录？
2. 搜索起点、深度和文件系统边界是否准确？
3. 模式属于 `glob`、固定字符串、`BRE` 还是 `ERE`？
4. 一条记录在哪里结束，字段又怎样拆分？
5. 特殊文件名会不会被空白或换行拆坏？
6. 预览集合与执行集合是否来自同一筛选表达式？
7. 当前证据能证明什么，又不能证明什么？
8. 下一条最有区分度的证据是什么？

01限定搜索空间

02建立对象谓词

03预览候选集合

04选择记录与字段

05构造安全 `argv`

06执行分层验收

第五章 文件查找、文本筛选与批量处理

系统管理员面对的通常不是一个已经知道路径的文件，而是一组随目录树变化的对象：先按路径、类型、所有者、大小或时间找到候选文件，再从文件内容中选择记录、拆分字段、排序聚合，最后才可能执行修改。最常见的误判，是把这几个对象层混成一层：用文件名模式代替内容匹配、把按行输出当作任意文件名协议、在未确认候选集合时直接使用 `sed -i`，或者看到子命令退出为零就认为所有目标都已经达到终态。

本章采用“文件集合 → 文本记录 → 字段和键 → 参数向量 → 批量副作用 → 分层验证”的主线。前一章已经建立路径、文件类型和链接的基础；本章只引用这些对象，不重新展开路径解析。Shell 展开与引用留在第 01 章，复制、归档、压缩和远程传输留给第 06 章；`sed` 与 `awk` 只讲完本章任务所需的选择、替换、字段和小型聚合，不扩展成完整编程教材。

本章的核心判断链 先证明搜索范围 → 再证明候选集合 → 再证明记录与字段合同 → 最后执行副作用并重新查询终态

概念

文件集合 是在某一搜索空间和某组条件下成立的对象集合。它不是一份永久不变的路径清单：目录树可能变化、权限可能阻止遍历、链接策略可能改变观察对象。判断集合时必须同时保留搜索起点、表达式、标准错误和生成时间；仅看到若干输出路径，不能证明没有遗漏。

概念

`find` 表达式 是对每个遍历对象求值的条件与动作组合，由测试、逻辑运算符和动作共同构成。相邻测试默认使用 AND，AND 的优先级高于 OR，动作本身也有真值并参与短路求值。因此，复杂命令必须先使用括号明确逻辑，再把统一的预览或执行动作放到完整条件之后。

概念

正则表达式 描述文本记录中的字符模式，不等同于 Shell glob。`\*.conf` 是路径名称模式，`\*\.conf\$` 才是正则形式；`grep -F` 又把模式视为固定字符串。选择错误的模式语言，会让命令看似合理却得到过宽、过窄或完全不同的集合。

概念

记录与字段 是文本处理的两层合同。记录边界先回答“一条输入在哪里结束”，字段边界再回答“记录内部怎样拆列”。`grep`、`sed` 默认按换行处理记录，`awk` 默认一行一条记录并根据 `FS` 拆字段。若记录边界已经破坏对象，后续字段工具无法把它恢复。

概念

NUL 分隔 利用 Linux 文件名不能包含 NUL 字节这一事实，把任意路径名称作为完整记录传递。`find -print0`、`grep -Z` 等生产端必须与 `xargs -0` 等消费端成对出现。NUL 解决的是名称分隔问题，不会自动解决目录树在“检查后、使用前”发生变化的竞态。

概念

批量执行边界 是从“我找到了什么”切换到“我要改变什么”的分界线。安全流程把预览、执行和验收拆开：预览证明集合正确，执行把每个路径作为独立 argv 传给子命令，验收重新查询新状态、旧状态、反例、备份和错误流。命令成功只证明命令按自身约定结束，不能替代终态验证。

操作语义速查

这里先建立关键命令的接口地图。正文专题会继续解释为什么这样选择、怎样验证以及在哪些边界下不能直接执行。

find

SYNOPSIS

```
find [OPTIONS] [START...] [EXPRESSION]
```

实时遍历一个或多个起点，对每个对象求值表达式，并通过动作输出或传递匹配对象。

重要参数 / 形式

START...

明确搜索起点；关键变更优先使用绝对路径。

`-type` / `-name` / `-user` / `-size` / `-mtime`

把任务条件翻译为可解释的对象测试。

`\( A -o B \) / ! TEST`

显式表达 OR、分组和否定，避免优先级误判。

`-print` / `-printf`

用于可读预览和审计输出。

`-print0`

以 NUL 终止每个路径，供支持 NUL 的下游消费。

`-exec command -- {} +`

由 `find` 直接把多个路径构造成独立参数。

locate / updatedb

SYNOPSIS

```
locate [OPTIONS] PATTERN...
updatedb [OPTIONS]
```

`locate` 查询预先建立的名称索引；`updatedb` 重新扫描允许的目录并更新索引。结果速度快，但时效性低于实时遍历。

重要参数 / 形式

`locate PATTERN`

快速生成路径候选，不直接证明对象当前存在。

`-i`

名称匹配时忽略大小写。

`-n N`

只显示前 N 个匹配，适合快速抽样。

`updatedb`

更新索引；排除规则和权限仍决定哪些路径可被收录。

`stat -- PATH`

把索引候选带回实时证据。

grep

SYNOPSIS

```
grep [OPTIONS] PATTERN [FILE...]
```

从文本记录中选择与模式匹配的记录；模式引擎和输出形式会直接改变结果语义。

重要参数 / 形式

`-F`

把模式当作固定字符串，避免不需要的正则解释。

`-E`

使用扩展正则表达式。

`-r / -R`

递归读取目录；符号链接处理边界不同，使用前需确认。

`-n / -i / -v`

显示行号、忽略大小写、反选。

`-l / -L / -c / -q`

输出匹配文件名、不匹配文件名、计数或仅使用退出状态。

`cut / sort / uniq / tr / wc`

SYNOPSIS

```
cut OPTION... [FILE...]  
sort [OPTION]... [FILE]...  
uniq [OPTION]... [INPUT [OUTPUT]]  
tr [OPTION]... STRING1 [STRING2]  
wc [OPTION]... [FILE]...
```

这些工具适合对合同明确的记录流执行字段提取、排序、相邻去重、字符转换和计数。

重要参数 / 形式

```
cut -d DELIM -f LIST
```

按单字符分隔符提取固定字段。

```
sort -t DELIM -k KEY / -n / -r
```

明确字段分隔、排序键、数字比较和方向。

```
sort ... | uniq -c
```

先让相同记录相邻，再计数；排序会改变原始顺序。

```
tr -d / -s
```

删除字符集合或压缩重复字符，不做字符串子串替换。

```
wc -l / -c / -w
```

统计换行、字节或单词；数字必须解释成明确对象。

sed

SYNOPSIS

```
sed [OPTIONS] SCRIPT [INPUTFILE...]
```

按地址选择记录并执行替换、打印或删除等编辑动作。默认先把结果写到标准输出，再决定是否修改文件。

重要参数 / 形式

```
-n 'ADDRESS p'
```

关闭默认输出，只打印明确选择的记录。

```
-E
```

在脚本中使用扩展正则表达式。

```
s/OLD/NEW/g
```

替换一条记录中的全部匹配；未加 **g** 时只替换第一个。

```
-e SCRIPT / -f FILE
```

组合多个脚本或从脚本文件加载规则。

```
-i.SUFFIX
```

原地替换并保留备份；执行前仍应先预览。

awk

SYNOPSIS

```
awk [OPTIONS] 'PATTERN { ACTION }' [FILE...]
```

把输入解释为记录与字段，根据条件执行打印、计算和小型聚合。

重要参数 / 形式

-F DELIM

设置输入字段分隔符。

-v NAME=VALUE

在处理输入前安全传入外部值。

\$0 / \$1... / NF

整条记录、字段和当前字段数。

NR / FNR

全部输入累计记录号与当前文件记录号。

BEGIN / END / OFS

初始化、最终汇总和稳定输出字段合同。

xargs

SYNOPSIS

```
xargs [OPTIONS] [COMMAND [INITIAL-ARGS]]
```

从输入项目构造命令参数并分批执行。它处理的是 `argv`，不应被用来把路径重新拼成 Shell 代码。

重要参数 / 形式

-0

按 NUL 读取输入，必须与 NUL 生产端配对。

-r

GNU 扩展：输入为空时不运行命令。

-n N

限制每次调用使用的输入项目数。

-I REPL

占位替换模式，适合参数位置不能简单追加的场景，但通常降低批量效率。

--

在被调用命令支持时结束其选项解析，保护以连字符开头的路径。

[知识专题] 从目录树对象到记录流：先确定自己正在处理什么

同一条流水线里可能同时出现文件对象、路径文本、文本行、字段和命令参数。它们看起来都能在终端上显示为字符串，但安全边界完全不同。理解这些层次，是后续所有工具选择的前提。

① [知识点] 文件对象、路径名和内容是三个筛选维度

`find` 主要依据目录树和元数据选择文件对象，例如类型、所有者、权限、大小和时间；`grep` 依据文件内容选择记录；文件名模式则只筛选路径名称。扩展名并不决定 Linux 文件类型，名称中含有 `.conf` 也不保证内容是配置文本。

稳定的判断顺序通常是：

```
限定目录树和文件类型
→ 按所有者、名称、大小、时间等缩小对象集合
→ 再读取候选文件内容
→ 按文本模式继续筛选
```

这样既能减少无关 I/O，也能避免把目录、设备或符号链接目标误当成普通文本文件处理。

② [知识点] 记录边界先于字段边界

一条记录在哪里结束，决定了后续工具能否保持对象完整。文本日志通常使用换行分隔记录；冒号、逗号或制表符可能用于记录内部字段。文件名集合则不能默认使用换行，因为文件名本身可以包含换行。

```
记录边界：一条输入何时结束
字段边界：一条记录内部怎样拆列
```

`cut -d: -f1` 只有在每条记录确实使用冒号分隔时才可靠。`awk -F:` 也依赖同一结构假设。面对面向人的、按空格对齐的输出，不应仅凭“看起来像第十列”建立长期脚本。

③ [知识点] 路径列表最终要变成独立 argv，而不是重新拼成 Shell 代码

外部命令接收的是参数向量。路径中的空格不应该被再次拆分，引号字符也不应该被解释为新的 Shell 语法。因此批量处理的目标是把每个路径作为一个独立参数传给子命令，而不是把所有名称拼成一行，再交给 `sh -c` 猜测。

安全接口包括：

```
find START TESTS -exec command -- {} +
find START TESTS -print0 | xargs -0 -r command --
```

第一种由 `find` 直接构造参数；第二种要求生产端和消费端都使用 NUL 协议。

④ [知识点] 本章的状态模型是“集合状态 + 转换状态 + 副作用状态”

需要分别验证：

- 搜索范围是否正确；
- 候选集合是否完整且不过宽；
- 文本转换是否保持了记录结构；

- 批量命令是否收到正确 `argv`;
- 每个目标是否达到终态;
- 应被排除的对象是否保持不变。

[Cheatsheet] `find` 选对象, `grep` 选记录, 字段工具拆列, `xargs` 构造 `argv`; 先确定记录分隔符, 再确定字段分隔符; 任意文件名集合优先使用 `NUL`。

[知识专题] `find` 的搜索空间：起点、遍历和可见性

`find` 的条件写得再精确, 如果搜索起点和遍历边界不对, 结果仍然会为空、过宽或跨入不应处理的文件系统。调查 `find` 命令时, 首先拆开阅读“从哪里开始”和“怎样遍历”, 然后再看测试和动作。

① [知识点] 起点决定可见对象集合

基本形式是:

```
find START... EXPRESSION
```

可以提供一个或多个起点。省略起点时, GNU `find` 从当前目录 `.` 开始。相对起点依赖当前工作目录; 绝对起点不依赖当前目录。考试和生产变更中, 建议把关键起点写清楚, 避免因切换目录而改变搜索范围。

```
find /etc /usr/local/etc -type f -name '*.conf'
find . -maxdepth 1 -type f
```

传给 `-name` 的通配模式应由引号保护:

```
find /etc -name '*.conf'      # 模式由 find 解释
find /etc -name *.conf        # Shell 可能提前展开, 语义依赖当前目录
```

② [知识点] 深度选项限制“进入多深”, 不替代其他条件

常见边界:

- `-maxdepth 0`: 只检查起点本身;
- `-maxdepth 1`: 检查起点和直接成员;
- `-mindepth 1`: 不让起点本身参与匹配;
- `-xdev` 或 `-mount`: 不下降到其他文件系统。

```
find /srv/data -mindepth 1 -maxdepth 1 -type f
find / -xdev -type f -size +1G
```

`-xdev` 限制的是目录遍历跨文件系统，不代表结果只属于某种设备类型。它在从 `/` 开始调查大文件时特别重要，可避免进入 `/proc`、网络挂载和其他独立文件系统。

③ [知识点] 符号链接跟随策略改变被观察的对象

GNU `find` 默认采用 `-P`：遍历时不跟随符号链接，测试看到的是链接本身。`-L` 会跟随符号链接，可能进入链接指向的目录树，并改变 `-type` 等测试的观察对象；`-H` 只对命令行起点中的符号链接作特殊处理。

本章默认不使用 `-L` 执行批量修改。需要跟随链接时，必须先明确：

链接本身是否是目标
→ 目标对象是否在允许范围内
→ 是否可能形成循环或跨越边界
→ 修改后怎样证明没有触及额外对象

④ [知识点] 权限错误和空集合不是同一种证据

遍历目录通常需要执行权限，读取目录成员名称还需要读取权限。`find` 可能同时输出部分结果并在标准错误中报告 `Permission denied`。如果只看标准输出为空，无法区分：

- 真正没有匹配；
- 起点不存在；
- 无权进入子目录；
- 条件写错；
- 错误被重定向后忽略。

调查时保留 `stderr`，必要时分开记录：

```
find /srv -type f -name '*.conf' \  
> /tmp/find.out 2> /tmp/find.err
```

[Cheatsheet] 先看起点，再看 `-maxdepth/-mindepth/-xdev`；默认 `-P` 不跟随遍历中的符号链接；标准输出为空不能证明没有对象，必须同时检查 `stderr` 和退出状态。

[操作专题] 用 `find` 测试把自然语言要求翻译为对象谓词

稳定的 `find` 命令应让每个要求对应一个可解释测试。先逐项验证测试，再组合逻辑，最后才添加动作。不要一开始就把长表达式和破坏性命令写在一起。

① [操作] 名称、路径和文件类型

作用对象：遍历到的当前路径。

基本形式：

```
find START -type f -name '*.conf'
find START -path '*/conf.d/*.conf'
```

关键边界：

- `-name` 只匹配 basename，不含前导目录；
- `-path` 匹配从起点产生的完整路径文本；
- `-iname` 执行不区分大小写的名称匹配；
- `-type f/d/l/b/c/p/s` 分别筛选常规文件、目录、符号链接、块设备、字符设备、FIFO 和套接字。

```
find /etc -type f -name '*.conf'
find /srv -type f -path '*/conf.d/*.conf'
find /dev -type b
```

名称模式由 `find` 解释，不是正则表达式。`*.conf` 中的 `*` 匹配任意字符序列；它不会自动表达“文件内容中的任意文本”。

② [操作] 所有者、组和权限

典型形式：

```
find /var -user root -group mail
find /srv -nouser -o -nogroup
find /home -type f -perm -002
```

权限测试必须区分：

- `-perm MODE`：模式精确匹配；
- `-perm -MODE`：指定的所有权限位都必须存在；
- `-perm /MODE`：指定权限位中至少一个存在。

```
find /srv -type f -perm 0640      # 精确为 0640
find /srv -type f -perm -0600    # 至少具备用户读写
find /srv -type f -perm /002     # 任意目标中 other 写位存在
```

`-perm` 是筛选接口。完整的传统权限、umask 和特殊权限位属于《传统权限、umask 与特殊权限位》一章，本章只解释完成筛选所需语义。

③ [操作] 大小条件中的单位和正负号

基本形式：

```
find /usr/share -type f -size +52428800c -size -104857600c
```

常用单位包括 **c** 字节、**k** KiB、**M** MiB、**G** GiB。**+n** 表示大于 n 个单位，**-n** 表示小于 n 个单位，无符号形式按单位块取整匹配。由于取整存在，不能把 **-size 1M** 简化为“精确等于 1 MiB”。需要精确边界时用 **stat -c %s** 抽查字节数。

```
find /srv -type f -size +10M
find /srv -type f -size -1048576c
stat -c '%s %n' /srv/example
```

④ [操作] 修改时间和参照文件

-mmin 按分钟计数，**-mtime** 按完整 24 小时周期计数：

```
find /home/student -type f -mmin -120
find /var/log -type f -mtime +7
```

通用数值语义：

- **-n**：少于 n 个计数单位；
- **+n**：多于 n 个计数单位；
- **n**：取整后正好为 n。

-newer REFERENCE 通过参照文件比较修改时间，适合表达“某个标记文件之后”：

```
touch -d '2026-07-12 09:00:00' /tmp/cutoff
find /srv -type f -newer /tmp/cutoff
```

静态任务中应清楚记录参照文件的来源，不能编造考试环境时间戳。

⑤ [验证] 对边界对象进行 **stat** 抽查

find 结果只说明测试在遍历时成立。对时间和大小的边界值，抽取最接近上下界的对象并使用 **stat** 验证：

```
find /srv -type f -size +50M -printf '%s\t%TY-%Tm-%Td %TH:%TM\t%p\n' |
sort -n | head
```

-printf 是 GNU **find** 的格式化输出，适合审计；它不自动添加换行，格式中需要明确写 **\n**。

[Cheatsheet] **-name** 看 basename，**-path** 看路径；**-perm MODE/-MODE//MODE** 分别是精确、全部位、任一位；大小和时间数值有取整；关键边界用 **stat** 抽查。

[操作专题] `find` 表达式、逻辑和动作：让命令可以被逐项证明

`find` 表达式由选项、测试、动作和运算符组成。它对每个遍历对象从左向右求值，并使用短路逻辑。复杂命令的排错关键是把表达式拆成可验证的片段。

① [操作] 隐式 AND、OR、NOT 和括号

相邻测试之间默认是 AND：

```
find /var -type f -user root -group mail
```

等价于：

```
find /var -type f -a -user root -a -group mail
```

AND 的优先级高于 OR。下面命令并不表示“普通文件且名称为 `.conf` 或 `.ini`”：

```
find /etc -type f -name '*.conf' -o -name '*.ini'
```

它会右侧 `.ini` 分支绕过 `-type f`。正确写法显式分组：

```
find /etc -type f \( -name '*.conf' -o -name '*.ini' \)
```

NOT 可写为 `!` 或 `-not`，Shell 中的 `!` 应避免被历史展开干扰，常用反斜杠或单引号保护上下文。

② [知识点] 短路求值决定动作是否执行

对于 `A -a B`，如果 A 为假，则 B 不再求值；对于 `A -o B`，如果 A 为真，则 B 不再求值。动作也有返回真值，因此把 `-print` 放在 OR 表达式某一侧会改变哪些对象被显示。

稳定做法是：

```
先用完整测试形成候选  
→ 再在表达式末尾放统一动作
```

```
find /etc -type f \( -name '*.conf' -o -name '*.ini' \) -print
```

③ [操作] 使用 `-print`、`-printf` 和 `-ls` 预览

没有显式动作时，GNU `find` 通常隐式使用 `-print`。为了审查集合，建议显式写出动作，并根据需求选择格式：

```
find /srv -type f -name '*.conf' -print
find /srv -type f -name '*.conf' \
    -printf '%m\t%u:%g\t%s\t%TY-%Tm-%Td %TH:%TM\t%p\n'
find /srv -type f -name '*.conf' -ls
```

面向人的预览可以使用换行；准备交给机器处理任意文件名时应使用 `-print0`。

④ [操作] `-exec ... {} \;` 与 `-exec ... {} +`

```
find /srv -type f -name '*.conf' -exec stat -- {} \;
find /srv -type f -name '*.conf' -exec stat -- {} +
```

- `{}` `\;`：通常每个匹配对象执行一次命令；
- `{}` `+`：尽可能批量把多个路径作为参数，减少进程启动；
- `--`：在支持该约定的子命令中结束选项，防止路径被当作选项；
- `{}`：由 `find` 替换为完整参数，不需要通过文本分词。

`-execdir` 在匹配对象所在子目录执行命令，可减少部分路径竞态和选项注入风险，但它改变工作目录，并要求认真处理 `PATH`。本章把它作为安全边界和选择题，不把它描述为万能保护。

⑤ [操作] 交互确认不是批量变更的完整验收

`-ok` 或 `-okdir` 会逐项询问确认，适合少量临时操作，但不能替代候选清单、审计记录和执行后验证。大量对象逐项输入 `y` 容易产生疲劳误判，也不利于复现。

[Cheatsheet] `AND` 高于 `OR`，复杂 `OR` 必须括号；动作也参与逻辑；预览优先 `-print/-printf`；批量参数优先 `-exec ... {} +`，逐个执行才使用 `\;`。

[知识专题] `locate` 与 `updatedb`：快速结果为什么不等于实时事实

`locate` 通过名称数据库快速返回候选路径，适合“我大概知道文件叫什么，但不知道在哪”的发现阶段。它不遍历当前目录树，因此速度快，但证据强度低于实时查询。

① [知识点] `locate` 查询的是上次索引时的名称状态

```
locate logrotate.conf
locate -i networkmanager.conf
locate -n 5 passwd
```

常用选项：

- `-i`：忽略大小写；
- `-n N`：最多显示 `N` 个结果；

- 不同实现还可能提供只显示当前仍存在对象的选项，使用前应查看本机 `locate --help`。

刚创建的文件可能尚未进入数据库，已删除的文件也可能暂时仍在结果中。`locate` 未找到不能证明文件不存在。

② [知识点] `updatedb` 的排除规则决定哪些路径永远不被索引

管理员可运行：

```
sudo updatedb
```

这只说明发起了一次数据库更新，不代表所有挂载点、文件系统和目录都被纳入。实现可能根据文件系统类型、挂载选项或配置中的排除路径跳过目录。数据库具体路径和定时更新方式具有实现和版本差异，应通过本机软件包、手册页和配置确认，不把旧版 `mlocate` 路径当成 RHEL 9 永久常量。

③ [诊断点] 候选路径必须回到实时证据

```
locate app.conf |
head

stat -- /candidate/path
find /expected/root -type f -name 'app.conf'
```

诊断顺序：

```
locate 无结果或出现陈旧路径
→ 确认命令实现和数据库状态
→ 检查 updatedb 排除范围
→ 必要时更新数据库
→ 用 stat/find 确认当前对象
```

[Cheatsheet] `locate` 是名称索引，不是实时遍历；`updatedb` 更新索引，但受排除策略影响；最终对象必须用 `stat` 或限定范围的 `find` 确认。

[知识专题] 正则表达式与 `grep`：选择记录而不是猜字符串

正则表达式描述文本模式，Shell glob 描述路径名称模式。两者共享少数字符，但语义不能互换。写 `grep` 前先确定要匹配固定文本、基本正则还是扩展正则，并用引用阻止 Shell 解释模式。

① [知识点] glob、固定字符串、BRE 和 ERE 是不同语言

场景	入口	示例
路径名称通配	Shell 或 <code>find -name glob</code>	<code>*.conf</code>

场景	入口	示例
固定文本	<code>grep -F</code>	<code>server[1]</code>
基本正则 BRE	<code>grep</code>	<code>^root:</code>
扩展正则 ERE	<code>grep -E</code>	<code>^(sshd sudo):</code>

`grep -F` 不把 `.`、`[`、`*` 等视为正则元字符，搜索配置字面量时更安全。模式通常放在单引号中：

```
grep -F 'server[1]' file
grep -E '^(sshd|sudo):' file
```

② [知识点] 锚点、字符类和重复表达边界

常见 ERE：

- `^`、`$`：记录开头和结尾；
- `.`：任意单个字符；
- `*`、`+`、`?`：前一项重复 0 次以上、1 次以上、0 或 1 次；
- `{m,n}`：重复次数范围；
- `[...]`：字符集合；
- `[^...]`：否定字符集合；
- `(A|B)`：分组和选择；
- `[[[:space:]]]`、`[[[:digit:]]]`、`[[[:alpha:]]]`：POSIX 字符类。

匹配完整配置行时显式锚定：

```
grep -E '^[[:space:]]*Listen[[:space:]]+8080[[:space:]]*$$' file
```

它不会误选 `# Listen 8080`、`Listen 80800` 或包含额外字段的行。

③ [操作] 控制输出形式，而不是默认打印所有匹配行

```
grep -nE 'PATTERN' file      # 匹配行和行号
grep -vE 'PATTERN' file      # 反选
grep -cE 'PATTERN' file      # 每个输入的匹配行数
grep -lE 'PATTERN' files...   # 只显示有匹配的文件名
grep -LE 'PATTERN' files...   # 只显示没有匹配的文件名
grep -qE 'PATTERN' file      # 静默判断
grep -oE 'PATTERN' file      # 只输出匹配片段
```

递归搜索：

```
grep -rnE 'PATTERN' /etc/app
```

`-r` 和 `-R` 的符号链接处理边界不同，必须通过本机手册确认后再用于会跨链接的目录树。若已经用 `find` 精确限定文件类型和范围，通常不需要让 `grep` 再次自行遍历。

④ [知识点] 退出状态是三态证据

正常情况下：

- `0`：至少选择到一条记录；
- `1`：没有选择到记录；
- `2`：发生错误。

因此“无匹配”不是命令执行错误。脚本或流水线必须把 `1` 和 `2` 分开，尤其不能把错误文件、权限失败和真正无匹配混为一谈。`-q` 在发现匹配后可能提前退出，诊断输入错误时应谨慎使用。

⑤ [边界] locale 和二进制判断会改变结果

字符范围、大小写折叠和排序可能受 locale 影响。优先使用 POSIX 字符类；只有任务明确要求字节顺序时，才对局部命令设置 `LC_ALL=C`。`grep` 认为输入是二进制时可能只报告“binary file matches”；需要文本处理前先确认文件类型和内容，不把 `-a` 当成无条件默认。

[Cheatsheet] 字面量用 `-F`，ERE 用 `-E`；完整行用 `^...$`；输出行、文件名、数量或退出状态是不同接口；退出 `1` 是无匹配，`2` 才是错误。

[操作专题] `cut`、`sort`、`uniq`、`tr` 与 `wc`：小工具也要有明确输入合同

这些工具适合构造短而确定的记录流水线，但前提是输入结构明确。不要为了命令短而忽略记录分隔、字段分隔、排序规则和计数对象。

① [操作] 用 `cut` 提取稳定分隔字段

```
cut -d: -f1,3 /etc/passwd
cut -d: -f1-3 file
cut -c1-20 file
```

- `-d CHAR`：指定单字符字段分隔符；
- `-f LIST`：选择字段；
- `-c LIST`：按字符位置选择；
- `-s`：不输出没有分隔符的记录。

`cut` 不理解 CSV 引号、转义和嵌套分隔符；复杂结构应使用对应解析器。面向人的对齐输出中连续空格数量不稳定，也不适合作为 `cut -d' '` 的长期输入接口。

② [操作] 用 `sort` 明确排序键和比较方式

```
sort file
sort -n file
sort -t: -k3,3n -k1,1 /etc/passwd
LC_ALL=C sort -k2,2nr -k1,1 report.tsv
```

常用选项：

- `-n`：数字比较；
- `-h`：理解常见可读大小后缀；
- `-r`：逆序；
- `-t CHAR`：字段分隔符；
- `-k START,END`：排序键范围；
- `-u`：排序后只保留唯一记录。

`sort` 默认受 locale 排序规则影响。任务要求完全确定的机器顺序时，局部设置 `LC_ALL=C`，不要改变整个登录环境。

③ [操作] `uniq` 只折叠相邻重复记录

```
sort userlist.txt | uniq
sort userlist.txt | uniq -c
uniq -d sorted.txt
uniq -u sorted.txt
```

`uniq` 不会在整个未排序文件中任意寻找重复，它只比较相邻记录。先 `sort` 会改变原顺序；如果业务要求保持首次出现顺序，应使用 `awk '!seen[$0]++'` 等显式状态方法，并说明内存和输入规模边界。

④ [操作] `tr` 处理字符集合，不处理字符串子串

```
tr '[:lower:]' '[:upper:]' < file
tr -d '\r' < windows.txt
tr -s '[:space:]' ' ' < file
```

`tr -d 'root'` 删除的是字符集合中的 `r`、`o`、`t`，不是删除单词 `root`。子串替换使用 `sed` 或 `awk`。`tr` 从标准输入读取数据，常位于管道中。

⑤ [操作] `wc` 的数字必须解释为明确对象

```
wc -l file      # 换行数量
wc -w file      # 按当前规则识别的单词数
wc -c file      # 字节数
wc -m file      # 字符数
```

最后一条记录没有换行时，`wc -l` 不会把它计为一个换行。因此“文件有多少业务记录”只有在记录合同为“一条记录一个换行且末尾完整”时才等于 `wc -l`。

[Cheatsheet] `cut` 适合稳定单字符分隔；`sort` 明确键、数字和 locale；`uniq` 只看相邻；`tr` 操作字符集合；`wc -l` 统计换行，不自动等于业务对象数。

[操作专题] `sed` 的选择和替换：先把结果写到标准输出

`sed` 逐条读取输入记录，在模式空间中执行脚本，然后默认输出处理后的记录。它适合地址选择、短替换和简单流编辑；原地修改是额外模式，不应成为首次试验的默认动作。

① [操作] 用地址选择记录

```
sed -n '10,20p' file
sed -n '/^BEGIN$/,/^END$/p' file
sed -n '/ERROR/p' file
```

- 单个行号选择一行；
- `ADDR1,ADDR2` 选择范围；
- `/REGEXP/` 选择匹配记录；
- `$` 表示最后一行；
- `-n` 关闭默认输出，`p` 只打印显式选中记录。

如果忘记 `-n` 又使用 `p`，匹配行通常会出现两次：一次来自默认输出，一次来自 `p` 命令。

② [操作] 使用 `s///` 替换并控制作用范围

```
sed -E 's/old/new/' file
sed -E 's/old/new/g' file
sed -E 's#^/opt/old#&/archive#' file
sed -E 's/^([:]+):([[:]+)$$/\2:\1/' file
```

- 默认只替换每条记录的第一次匹配；
- `g` 替换该记录内所有匹配；
- `&` 代表完整匹配文本；
- `\1`、`\2` 引用捕获组；
- 分隔符不必是 `/`，处理路径时可改用 `#` 或 `|`。

配置修改应使用锚点和字符类限制完整行，不要只写宽泛的 `s/8080/8081/`。

③ [操作] 多个脚本使用 `-e` 或脚本文件

```
sed -e '/^[[:space:]]*#/d' -e '/^[[:space:]]*$/d' file  
sed -f cleanup.sed file
```

脚本开始变长、包含多条相互依赖规则时，应保存为可审查文件，而不是继续堆叠在一行中。复杂解析或跨记录状态通常更适合 `awk`。

④ [边界] `-i` 是文件替换操作，需要备份和元数据检查

```
sed -i.rhcsa05.bak -E 's/^Listen 8080$/Listen 8081/' file
```

带后缀会在同目录创建备份；不带后缀不生成备份。`sed -n -i` 如果脚本没有显式输出，可能把文件写空。原地编辑通常通过临时文件替换原路径，还可能影响硬链接关系、ACL、扩展属性或 SELinux 上下文，真实环境应按目标文件类型验证。

安全流程：

```
stdout 预览  
→ diff 或样本核对  
→ 建立候选清单  
→ 使用明确备份后缀执行  
→ 验证内容、备份和元数据
```

[Cheatsheet] `-n` 配合 `p` 做选择；`s///g` 控制同一记录内多次替换；`&` 是完整匹配，`\1` 是捕获组；首次尝试不使用 `-i`，批量原地修改必须保留备份。

[操作专题] `awk` 的记录、字段、条件和小型聚合

`awk` 最适合“每条记录都有稳定字段，我需要按条件选择、计算并格式化输出”的任务。本章只建立 RHCSA 和日常运维所需的记录模型、条件动作和小型聚合，不扩展为完整编程教材。

① [知识点] 默认输入模型是记录和字段

```
awk -F: '{print $1, $3}' /etc/passwd
```

常用内建状态：

- `$0`：当前完整记录；
- `$1`、`$2` …：当前字段；
- `NF`：当前记录字段数；
- `NR`：所有输入累计记录号；

- `FNR`：当前文件内记录号；
- `FS`：输入字段分隔符；
- `OFS`：输出字段分隔符；
- `RS`、`ORS`：输入和输出记录分隔符。

```
awk -F: -v OFS='\t' '{print NR, $1, $3}' /etc/passwd
```

② [操作] 使用 `pattern { action }` 选择记录

```
awk -F: '$3 >= 1000 {print $1, $3}' /etc/passwd
awk -F: '$2 == "sshd" && $4 == "FAIL" {print $3}' events.log
awk '$0 ~ /ERROR/ {print NR, $0}' app.log
```

模式为真时执行动作；省略动作时默认打印整条记录；省略模式时对每条记录执行动作。正则匹配使用 `~`，否定匹配使用 `!~`。

字段数也是重要验证条件：

```
awk -F: 'NF != 5 {print FNR, $0 > "/dev/stderr"}' events.log
```

③ [操作] 使用变量和 `-v` 传递外部参数

```
awk -F: -v user="$target_user" '$1 == user {print $0}' /etc/passwd
```

`-v` 在 `awk` 开始处理前赋值，避免把 Shell 内容直接拼入 `awk` 源码。Shell 变量仍需正确引用。对来自不可信输入的动态正则，需要额外评估其是否应作为固定字符串处理。

④ [操作] 计数、求和和关联数组

```
awk -F: '
    $2 == "sshd" && $4 == "FAIL" {
        count[$3]++
        bytes[$3] += $5
    }
    END {
        for (user in count)
            print user, count[user], bytes[user]
    }
' events.log
```

`awk` 关联数组按键聚合，但遍历数组的顺序未必符合业务要求。需要确定性顺序时，把结果交给 `sort`，不要假设数组天然按用户名或插入顺序输出。

⑤ [操作] 用 `printf` 固定输出合同

```
awk -F: 'BEGIN {OFS="\t"}
{printf "%s\t%d\n", $1, $3}
' /etc/passwd
```

`print` 自动使用 `OFS/ORS`，`printf` 由格式字符串控制。后续还要排序或读取的结果，应明确使用制表符或其他稳定分隔符，并避免混入说明文字。

[Cheatsheet] `$0` 是记录，`$1` 是字段，`NF` 是字段数，`NR/FNR` 是记录号；条件写在动作前；外部变量用 `-v`；聚合数组输出顺序不确定，排序交给 `sort`。

[操作专题] 特殊文件名、NUL 和 `xargs`：保持每个路径都是一个参数

换行分隔的路径列表只在你能够证明文件名不含换行时才可靠。系统级批量修改默认不能做这个假设。NUL 协议解决记录边界问题，`xargs` 或 `find -exec` 解决参数构造问题。

① [知识点] 空格不是最难的文件名，换行才会破坏“每行一个路径”

合法文件名可以包含：

- 空格和制表符；
- 单引号、双引号和反斜杠；
- 通配符字符；
- 换行；
- 前导连字符。

不能包含的是 `/`（目录分隔符）和 NUL。`for f in $(find ...)`、`find | while read` 的默认形式、普通 `xargs` 都可能因空白分词或反斜杠处理破坏名称。

② [操作] 生产端和消费端必须使用同一种记录协议

```
find /srv -type f -print0 |
xargs -0 -r stat --
```

- `-print0`：每个路径后输出 NUL；
- `xargs -0`：只把 NUL 视为项目分隔符；
- `-r`：输入为空时不运行命令；
- `--`：结束子命令选项。

内容筛选后继续保持 NUL：

```
find /srv -type f -name '*.conf' -print0 |
  xargs -0 -r grep -lZ -E '^Listen[[:space:]]+8080$' -- |
  xargs -0 -r stat --
```

这里 `grep -Z` 让文件名输出以 NUL 结束。管道的每一段都必须检查是否保持了同一边界。

③ [操作] `xargs` 的批量大小和替换模式

```
xargs -0 -r -n 20 command --
xargs -0 -r -I '{}' command -- '{}'
```

- `-n N`：每次最多传 `N` 个项目；
- `-s SIZE`：限制构造命令行大小；
- `-I`：按输入记录替换占位符，通常意味着逐条或小批执行；
- `-P`：并行执行，可能打乱顺序并放大副作用，本章不把它作为修改任务的默认答案。

不要同时把 `-I` 当成高效批量接口。需要把所有路径附加到命令末尾时，普通 `xargs -0` 或 `-exec ... {} +` 更直接。

④ [比较] `-exec ... {} +` 与 `xargs -0` 的选择

场景	推荐入口
候选直接来自同一条 <code>find</code> ，子命令可接收多个路径	<code>-exec command -- {} +</code>
候选要经过 <code>grep -Z</code> 等多个 NUL 处理阶段	<code>xargs -0 -r</code>
需要在每个对象所在目录执行	评估 <code>-execdir</code>
需要逐项确认	少量对象可用 <code>-ok/-okdir</code>
需要并行修改	默认不做，先证明幂等性和顺序无关

⑤ [边界] NUL 解决名称分隔，但不消除目录树竞态

从预览到执行之间，文件可能被删除、替换、改名或更改所有者。对高风险目录和不可信用户可写目录，必须评估：

```
候选清单生成时间
→ 执行时对象身份是否仍相同
→ 子命令是否会跟随符号链接
→ 是否需要在更受控的目录或维护窗口操作
```

[Cheatsheet] 任意文件名集合使用 NUL；`-print0`、`grep -Z`、`xargs -0` 必须成对；空输入加 `-r`；直接来自 `find` 时优先 `-exec ... {} +`；NUL 不解决 TOCTOU 竞态。

[诊断专题] 结果为空、过多、错序或批量动作异常时怎样推进

文本流水线故障最容易诱发“继续加一个管道试试”。稳定诊断应把集合选择、记录转换和副作用分层，找到第一处证据偏离预期的位置。

① [诊断] `find` 结果为空或过少

症状：没有候选或明显少于预期

- 当前证据：起点、`stderr`、退出状态、最简单 `find` 输出
- 假设：起点错误、无遍历权限、模式被 `Shell` 展开、时间/大小边界取整
- 区分证据：移除动作和后半条件，逐个添加测试
- 最小修复：修正起点、引用、权限或边界
- 再验证：数量、边界对象、应纳入样本

```
find /srv -maxdepth 1 -print
find /srv -type f -print
find /srv -type f -name '*.conf' -print
```

② [诊断] 结果过多或包含错误类型

优先检查：

- `OR` 表达式是否缺少括号；
- `-name` 与 `-path` 是否混淆；
- 是否忘记 `-type f`；
- 是否因 `-L` 跟随链接进入额外目录；
- `grep` 是否没有锚定完整行；
- 反选 `-v` 是否作用在错误阶段。

验证不能只看正例，还要主动检查一个应被排除的反例。

③ [诊断] 排序、去重或计数不符合直觉

重复仍存在

- 检查是否只是非相邻重复
- 检查大小写、尾随空白和不可见字符
- 用 `sed -n l` 或 `od` 观察记录
- 决定是否先标准化、排序，再 `uniq`

数字错序通常来自文本比较；字段键过宽会把后续字段也纳入比较；`locale` 可能改变字符顺序。`wc -l` 少一条时检查最后一条记录是否缺少换行。

④ [诊断] 批量动作只处理部分文件

保留以下证据：

- 原始 NUL 候选清单；
- 可读转义预览；
- 子命令 stderr；
- 批次数和退出状态；
- 修改后重新生成的命中清单。

可能原因包括命令行被拆批、某批子命令失败、文件在执行前变化、权限不同、文件内容不再匹配。不要仅重复执行整个命令；先定位未完成对象的共同特征。

⑤ [验证] 采用“集合、内容、反例、备份、错误”验收矩阵

集合：执行对象与预览对象一致
内容：每个目标达到目标文本状态
反例：范围外对象保持原状
备份：需要回退的原始内容存在
错误：stderr 和退出状态没有被管道吞掉

[Cheatsheet] 诊断时移除副作用，从最简单查询逐步加条件；每一步既查正例也查反例；批量任务保留原始候选清单，不能只靠最终输出回忆处理了谁。

[经典任务] 安全筛选并批量修改配置文件

环境与当前状态

系统中存在目录：

```
/srv/app/conf.d/
```

目录树中包含普通 `.conf` 文件、子目录、符号链接以及名称中含空格、换行或前导连字符的文件。部分普通文件由 `appsvc` 用户拥有。文件内容可能包含：

```
Listen 8080
  Listen 8080
# Listen 8080
Listen 80800
```

目标终态

只处理同时满足以下条件的对象：

1. 位于 `/srv/app/conf.d/` 目录树；
2. 不跨入其他文件系统；
3. 是普通文件，不跟随并修改符号链接目标；

- 4. 所有者为 `appsvc`；
- 5. `basename` 以 `.conf` 结尾；
- 6. 小于 1 MiB；
- 7. 最近 7 个完整 24 小时计数范围内修改；
- 8. 包含完整有效配置行 `Listen 8080`，允许行首尾空白和字段间多个空白。

将完整有效行统一替换为：

```
Listen 8081
```

限制条件

- 不使用 `for f in $(find ...)`；
- 任意文件名必须保持完整；
- 批量修改前必须保存机器可读 NUL 清单和可读预览；
- 空集合时不得执行 `sed`；
- 每个修改文件保留 `.rhcsa05.bak` 备份；
- 不修改注释行、`Listen 80800` 或范围外文件；
- 不声称完成 `live test`，答案只给出推荐命令和静态验收方法。

验收证据

评分对象	证据
静态候选集合	NUL 清单、转义预览、数量
内容候选集合	只包含完整有效旧行的文件
特殊文件名	每个路径保持一个 NUL 记录
新内容	每个候选至少存在目标新行
旧内容	候选中不再存在完整有效旧行
备份	每个修改文件旁存在指定后缀备份
反例	注释、相似行和范围外文件未变
错误	<code>stderr</code> 、子命令失败和空输入均可区分

[参考解答] 安全筛选并批量修改配置文件

以下命令是静态设计，未在本会话控制的 RHEL 9 虚拟机中执行。真实环境应先在副本或维护窗口验证。

① 调查：建立基线并确认目录边界

```
find /srv/app/conf.d -xdev -maxdepth 2 \
  -printf '%y\t%u:%g\t%s\t%p\n' |
  head -n 50
```

目的不是立刻得到最终集合，而是确认起点、层次、对象类型和所有者分布。高风险目录中还应检查是否存在其他用户可写目录和符号链接。

② 只按元数据生成静态候选 NUL 清单

```
find /srv/app/conf.d \
  -xdev \
  -type f \
  -user appsvc \
  -name '*.conf' \
  -size -1048576c \
  -mtime -7 \
  -print0 \
  > /root/rhcsa05-static-candidates.nul
```

参数含义：

- `-xdev`：不下降到其他文件系统；
- `-type f`：只处理普通文件；默认 `-P` 不跟随遍历中的符号链接；
- `-size -1048576c`：严格小于 1,048,576 字节；使用 `c` 避免 MiB 单位向上取整造成边界误判；
- `-mtime -7`：最近 7 个完整 24 小时计数范围；
- `-print0`：保存任意文件名。

空集合数量可用不会破坏名称的方式计算：

```
xargs -0 -r -n 1 printf '.\n' \
  < /root/rhcsa05-static-candidates.nul |
  wc -l
```

可读预览使用转义表示，不把它重新作为机器输入：

```
xargs -0 -r -n 1 printf '%q\n' \
  < /root/rhcsa05-static-candidates.nul \
  > /root/rhcsa05-static-preview.txt

less /root/rhcsa05-static-preview.txt
```

③ 按内容继续筛选并保持 NUL 协议

```
xargs -0 -r \  
  grep -lZ -E \  
    '^[[[:space:]]*Listen[[[:space:]]+8080[[[:space:]]*$$' -- \  
  < /root/rhcsa05-static-candidates.nul \  
  > /root/rhcsa05-content-candidates.nul
```

这里绝不能使用只搜索 8080 的宽泛模式。grep -lZ 每个匹配文件只输出一次，并以 NUL 终止名称。

再次生成可读预览和数量：

```
xargs -0 -r -n 1 printf '%q\n' \  
  < /root/rhcsa05-content-candidates.nul \  
  > /root/rhcsa05-content-preview.txt  
  
xargs -0 -r -n 1 printf '.\n' \  
  < /root/rhcsa05-content-candidates.nul |  
  wc -l
```

④ 在不修改文件的情况下预览替换结果

抽取一到数个候选，先运行不带 -i 的 sed：

```
xargs -0 -r -n 1 \  
  sed -n -E \  
    '/^[[[:space:]]*Listen[[[:space:]]+8080[[[:space:]]*$/p' \  
  -- \  
  < /root/rhcsa05-content-candidates.nul
```

随后对代表性文件比较转换前后：

```
sed -E \  
  's/^[[[:space:]]*Listen[[[:space:]]+8080[[[:space:]]*$/Listen 8081/' \  
  -- /path/to/representative.conf
```

确认不会改变注释、其他端口或额外字段后再执行。

⑤ 批量修改并保留备份

```
xargs -0 -r \  
  sed -i.rhcsa05.bak -E \  
    's/^[[[:space:]]*Listen[[[:space:]]+8080[[[:space:]]*$/Listen 8081/' \  
  -- \  
  < /root/rhcsa05-content-candidates.nul
```

-r 保证候选为空时不运行 sed。NUL 清单是执行对象的审计真源，不要在执行前改成普通换行列表。

⑥ 分层验证

验证每个候选包含新行：

```
xargs -0 -r \  
  grep -lZ -E \  
    '^[[[:space:]]*Listen[[[:space:]]+8081[[[:space:]]]*$' -- \  
  < /root/rhcsa05-content-candidates.nul \  
  > /root/rhcsa05-verified-new.nul  
  
cmp /root/rhcsa05-content-candidates.nul \  
    /root/rhcsa05-verified-new.nul
```

串行 `xargs` 下，若每个输入文件都输出一次且顺序未改变，两份 NUL 清单应一致。真实环境还应检查 `grep/xargs` 的 `stderr` 和退出状态。

检查旧行残留：

```
xargs -0 -r \  
  awk '  
    /^[[:space:]]*Listen[[[:space:]]+8080[[[:space:]]]*$/ {  
      printf "%s%c", FILENAME, 0  
      nextfile  
    }  
  ' \  
  -- \  
  < /root/rhcsa05-content-candidates.nul \  
  > /root/rhcsa05-old-remains.nul  
  
test ! -s /root/rhcsa05-old-remains.nul
```

检查备份：

```
xargs -0 -r -n 1 \  
  sh -c 'test -f "$1.rhcsa05.bak" || { printf "missing backup: %q\n" "$1" >&2; exit 1; }'  
 \  
  sh \  
  < /root/rhcsa05-content-candidates.nul
```

检查范围外反例：对预先记录的注释行、`Listen 80800` 和非 `appsvc` 文件重新计算内容校验或精确搜索，确认未变化。

⑦ 典型错误

- `find ... -name *.conf`：模式可能被当前 Shell 提前展开；
- `find ... | xargs sed -i`：空白和换行会拆坏文件名；
- `grep -l '8080'`：会选中注释和相似值；
- 预览一套条件、执行时重写另一套条件：集合可能漂移；

- `sed -i` 不带备份：失去最小回退材料；
- 只验证新行存在：不能证明旧行已消失，也不能证明范围外对象未变。

[经典任务] 从结构化事件日志生成可审计聚合报告

环境与输入合同

输入文件：

```
/srv/audit/events.log
```

有效记录使用冒号分隔五个字段：

```
timestamp:service:user:result:bytes
```

文件还可能包含注释行、空行、字段数错误、非数字字节字段和其他服务记录。

目标终态

1. 注释和空行不进入统计；
2. 非 5 字段或字节字段非数字的记录写入异常清单，并带原始行号；
3. 只统计服务为 `sshd` 或 `sudo` 且结果为 `FAIL` 的有效记录；
4. 按用户输出失败次数和字节总量；
5. 先按失败次数数字降序，再按用户名确定性排序；
6. 报告使用制表符分隔；
7. 单独输出总失败记录数和总字节数；
8. 原始文件不得修改。

验收关系

```
所有非注释、非空输入
= 结构有效记录 + 异常记录
```

```
总失败记录数
= 各用户失败次数之和
```

```
总失败字节数
= 各用户字节总量之和
```

[参考解答] 从结构化事件日志生成可审计聚合报告

① 建立输出文件并避免残留旧结果

```
report=/srv/audit/failures.tsv
invalid=/srv/audit/events.invalid
summary=/srv/audit/events.summary

: > "$report"
: > "$invalid"
: > "$summary"
```

显式清空可避免“本次没有异常记录，但旧异常文件仍然存在”的误判。

② 使用 `awk` 验证结构、筛选和聚合

```
awk -F: \
-v OFS='\t' \
-v invalid="$invalid" \
-v summary="$summary" '
    /^[[:space:]]*#/ || /^[[:space:]]*$/ { next }

    NF != 5 || $5 !~ /^[[:digit:]]+$/ {
        print FNR, $0 > invalid
        next
    }

    ($2 == "sshd" || $2 == "sudo") && $4 == "FAIL" {
        count[$3]++
        bytes[$3] += $5
        total_count++
        total_bytes += $5
    }

    END {
        for (user in count)
            print user, count[user], bytes[user]

        print "TOTAL", total_count + 0, total_bytes + 0 > summary
    }
' /srv/audit/events.log |
LC_ALL=C sort -k2,2nr -k1,1 > "$report"
```

关键语义：

- `FS=`：明确输入字段；
- `NF != 5` 和数字正则保护聚合；
- `OFS='\t'` 给下游稳定字段合同；
- 数组遍历顺序不可信，因此交给 `sort`；

- `LC_ALL=C` 只作用于本次排序；
- `+ 0` 让没有命中时仍输出数字 0。

③ 验证报告结构和排序

```
awk -F '\t' 'NF != 3 {print FNR, $0 > "/dev/stderr"; bad=1} END {exit bad}' \
"$report"

head "$report"
tail "$report"
cat "$summary"
cat "$invalid"
```

④ 验证汇总恒等式

```
awk -F '\t' '
{count += $2; bytes += $3}
END {print "TOTAL", count + 0, bytes + 0}
' "$report"
```

将此输出与 `events.summary` 比较。异常清单中的行号应能回到原始文件抽查。

⑤ 典型错误

- 使用 `cut -d: -f...` 后忽略字段数异常；
- 把 `bytes` 作为文本排序或拼接；
- 假设 `for (key in array)` 已按键排序；
- 直接把说明标题写入 TSV，破坏下游字段合同；
- 没有清空异常文件，导致旧错误混入本次结果；
- 只检查报告非空，没有验证总数恒等式和反例。

[本章收束] 把命令链升级为可证明的数据处理流程

本章建立的不是一组孤立命令，而是一条可以反复使用的工作方法：先把任务改写为对象条件，再固定并审查候选集合；只有记录和字段合同明确后，才执行转换或批量动作；最后用独立查询证明目标达到、旧状态消失、反例未变且错误没有被吞掉。

① [操作决策] 一条可执行的工作方法

- 限定搜索空间
- 把自然语言要求翻译为对象谓词
 - 只输出候选并检查 `stderr`
 - 检查数量、样本、边界对象和反例
 - 明确记录与字段分隔符
 - 选择固定字符串、正则或字段条件
 - 使用 `-exec ... {} +` 或完整 NUL 协议构造 `argv`
 - 执行最小副作用
 - 重新查询新状态、旧状态、反例、备份和错误流

② [验证] 章末检查清单

- ☐ 起点、深度、文件系统和链接策略已经明确；
- ☐ 文件对象条件与内容条件没有混用；
- ☐ glob、固定字符串、BRE 和 ERE 选择正确；
- ☐ 记录边界先于字段边界确定；
- ☐ 任意文件名没有通过空白分词或命令替换传递；
- ☐ 预览集合和执行集合来自同一筛选真源；
- ☐ 空集合不会意外执行批量命令；
- ☐ 新状态、旧状态、范围外反例和错误流均已检查；
- ☐ 没有把命令退出为零扩大为业务终态正确；
- ☐ 需要实机确认的版本或元数据行为已记录，而不是伪造输出。

③ [知识点] 主要判断表

需求或症状	首选入口	关键边界	下一层证据
实时选择目录树对象	<code>find</code>	起点、权限、深度、链接、表达式	<code>-printf</code> 、 <code>stderr</code> 、 <code>stat</code> 抽查
快速按名称找候选	<code>locate</code>	索引可能过期或排除路径	<code>stat</code> 或限定范围的 <code>find</code>
精确查找字面文本	<code>grep -F</code>	不需要正则时避免误解释	行号、文件名、计数与反例
按正则选择记录	<code>grep -E</code>	引用、锚点、字符类、locale	代表性匹配和退出状态
固定分隔字段提取	<code>cut</code>	只能依赖稳定单字符分隔符	字段数抽查
条件、计算或聚合	<code>awk</code>	<code>FS</code> 、 <code>NF</code> 、数字字段、数组无序	汇总恒等式与异常清单

需求或症状	首选入口	关键边界	下一层证据
排序后统计重复	<code>sort uniq -c</code>	<code>uniq</code> 只处理相邻重复，排序改变顺序	首尾样本与总数
选择后直接批量调用	<code>find ... -exec ... {} +</code>	候选来自 <code>find</code> ，参数通常追加在末尾	子命令状态与终态查询
任意上游传递路径	<code>... -print0 xargs -0 -r ...</code>	两端都必须使用 NUL	转义预览、空输入与批次检查
修改文本文件	<code>sed / awk</code>	先 <code>stdout</code> 预览，再备份修改	新旧模式、备份和反例

④ [边界] 本章能证明什么，不能证明什么

本章的查询和验证可以静态说明推荐命令、参数语义、数据边界与验收路径，但当前会话没有可控 RHEL 9 虚拟机，因此没有声称真实执行过这些任务。尤其是 `locate` 的具体实现与数据库路径、`sed -i` 对 ACL/SELinux 标签/硬链接关系的实际影响，以及大批量 `xargs` 的拆批和失败传播，需要在目标 RHEL 9 环境中再次观察。

⑤ [知识点] 向下一章交接

本章交付给第 06 章的是一个已经限定、预览并验证过的文件集合。下一章会讨论怎样复制、归档、压缩或远程传输这些对象，并继续验证目录结构、元数据、压缩格式和远端终态。本章不提前展开 `cp`、`tar`、`scp` 或 `rsync`；但下一章应复用这里建立的安全入口：不要在未经审查的路径集合上直接执行复制或归档。

本章总 Cheatsheet

```
# 实时查找与审计预览
find START TESTS -print
find START TESTS -printf 'FORMAT'

# 复杂逻辑
find START -type f \( -name '*.conf' -o -name '*.ini' \) -print

# 直接安全批量传参
find START TESTS -exec command -- {} +

# NUL 流水线
find START TESTS -print0 | xargs -0 -r command --

# 固定字符串与正则选择
grep -F 'literal text' file
grep -E '^[[:space:]]*KEY[[:space:]]+VALUE[[:space:]]*$' file

# 字段与聚合
awk -F: -v OFS='\t' 'CONDITION {print $1, $3}' file |
    LC_ALL=C sort -k2,2n

# sed 先预览, 后备份修改
sed -E 's/OLD/NEW/' file
sed -i.bak -E 's/OLD/NEW/' file
```